

FoodOnTheGo

Local Development Configuration and Android APK Review Guide

Web application first • Android APK second • Manual approval before any live deployment

NEW DELIVERY RULE

All development and testing must happen locally first. No production server, live database, live API keys, or cloud deployment may be changed during the local phase. iOS is explicitly out of scope until re-enabled.

Priority	Platform	Current Rule
1	Customer Web	Must run on localhost and be manually verified before any live upload.
2	Android	Must run locally, then produce an installable APK for manual testing on a real phone.
3	Backend/API	Laravel + MySQL + Redis must run locally and use a dedicated local database.
4	iOS	Skipped for the current phase. Do not spend time on iOS builds or testing.
5	Live/Cloud	Blocked until local Web + Android are approved by the project owner.

Document purpose: establish one repeatable local environment so every feature can be built, tested, visually inspected, and manually approved before deployment.

1. Local-First Operating Model

Local gate

No feature is “complete” because code exists. It is complete only after local API + local Web + local Android are connected, tested, visually inspected, and evidenced. Live deployment is a separate later phase.

Stage	Required result	Gate
A. Environment	Backend, database, Redis, Web and Android toolchain are installed and isolated.	Must pass before feature work.
B. Local Web	Customer Web runs on localhost and uses local API/database.	Manual browser review required.
C. Local Android	Flutter app uses local API and passes emulator/device testing.	Manual device review required.
D. APK	A versioned APK is produced, installed on the owner’s phone and manually checked.	Owner approval required.
E. Live deployment	Only after A-D pass.	Separate explicit deployment prompt.

- **iOS:** skip completely for now. Do not let iOS failures block Web/Android local work.
- **Production:** do not SSH, migrate, upload, seed, cache-clear, or change configuration on the live server during local development.
- **Credentials:** use local/test provider keys. Never copy production database or payment secrets into local configuration.
- **Evidence:** every module must show localhost URL, Android build, test results, screenshots, console/log review, and unresolved issues.

2. Target Local Architecture

Windows Development PC

```
|
+-- Laravel API ..... http://127.0.0.1:8000
| +-- MySQL local DB ..... 127.0.0.1:3306 / foodonthego_local
| +-- Redis ..... 127.0.0.1:6379
| +-- Reverb (when needed) . ws://127.0.0.1:8080
| +-- Local storage ..... storage/app (not production S3)
|
+-- Customer Web ..... http://localhost:5173
| +-- API ..... http://127.0.0.1:8000/api/v1
|
```

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

```

+-- Restaurant Web ..... http://localhost:5174 (when worked on)
+-- Admin Web ..... http://localhost:5175 (when worked on)
|
+-- Android Emulator
|   +-- API ..... http://10.0.2.2:8000/api/v1
|
+-- Physical Android Phone
    +-- Option A: http://<PC-LAN-IP>:8000/api/v1 (same Wi-Fi)
    +-- Option B: adb reverse tcp:8000 tcp:8000 (USB debugging)

```

Important Android networking rule

Inside a physical phone or Android emulator, localhost/127.0.0.1 normally refers to that Android device - not the Windows PC. Use 10.0.2.2 for the standard Android emulator, the PC LAN IP for a Wi-Fi phone, or adb reverse for a USB-connected phone.

3. Recommended Windows Development Stack

Component	Recommended local approach	Version rule
Git	Native Windows Git	Use project-compatible current stable.
PHP	Native PHP or project-provided runtime	Respect composer.json; Laravel 12 requires a compatible PHP 8.2+ runtime.
Composer	Native Composer	Use composer.lock; do not update dependencies during setup.
Node.js	Native Node	Use .nvmrc / package.json engines / lockfile if present.
MySQL	Docker Desktop container or existing local MySQL	Match project major version; expected MySQL 8.
Redis	Docker Desktop container	Match project config.
Flutter	Native Windows Flutter SDK	Use project-pinned/stable version compatible with pubspec.lock.
Android Studio	Native Windows	Install Android SDK, platform tools and emulator.
JDK	Use Flutter/Android project requirement	Do not upgrade blindly; current Flutter projects commonly use JDK 17.

Component	Recommended local approach	Version rule
Browser	Chrome/Edge	Use DevTools for console/network/responsive tests.

Do not mix environments unnecessarily

For a Windows-first workflow, the simplest setup is usually: native PHP/Composer + native Node + native Flutter/Android Studio, with MySQL and Redis in Docker. If the repository already has a tested docker-compose setup, prefer that instead of inventing a second topology.

4. Preflight: Inspect the Repository Before Installing Anything

1. Clone/open the new-session working copy. Do not work directly against a production-mounted folder.
2. Inspect composer.json and composer.lock for PHP/Laravel requirements.
3. Inspect package.json + package-lock.json/yarn.lock/pnpm-lock.yaml for Web toolchain.
4. Inspect pubspec.yaml + pubspec.lock for Flutter dependencies.
5. Inspect android/gradle files for JDK/Gradle/Android SDK requirements.
6. Inspect existing Docker/docker-compose files before creating new infrastructure.
7. Inspect all .env.example files and identify which variables are required for local, test and production.
8. Record actual directories for backend, customer web, restaurant web, admin web and mobile.

```
git status
git rev-parse --short HEAD
php -v
composer --version
node --version
npm --version
flutter --version
flutter doctor -v
adb version
docker --version
docker compose version
```

5. Local Environment Safety

Setting	Local value / behavior	Must never happen
APP_ENV	local	production on developer PC by accident
APP_DEBUG	true locally only	true on production

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

Setting	Local value / behavior	Must never happen
DB_DATABASE	foodonthego_local	production database name/host
FILESYSTEM_DISK	local for initial local phase	production S3 without explicit need
MAIL / SMS	log/test provider	real bulk provider without approval
Razorpay	TEST mode only	live charges
Google APIs	dedicated dev/restricted keys	unrestricted production key
Firebase	dev project or disabled until module	production push project during local setup

- ☐ Create a fresh foodonthego_local database.
- ☐ Verify no local .env references the production DB hostname.
- ☐ Verify no local configuration references production S3 bucket names unless read-only and explicitly approved.
- ☐ Verify Razorpay key ID is TEST mode and secret remains backend-only.
- ☐ Verify local/test OTP cannot work when APP_ENV=production.
- ☐ Verify .env files are ignored by Git.
- ☐ Run a secret scan before committing setup changes.

6. MySQL and Redis - Local Services

If the repository already provides Docker Compose, use it. Otherwise a minimal local-only service stack can be created after confirming it will not conflict with project infrastructure.

Example only - prefer the repository's existing compose file.

```
docker compose up -d mysql redis
```

```
docker compose ps
```

Service	Host	Port	Local verification
MySQL	127.0.0.1	3306	Connect and confirm foodonthego_local exists.
Redis	127.0.0.1	6379	redis-cli ping -> PONG

7. Laravel Backend - Local Configuration

9. Open the backend directory identified during repository audit.
10. Run composer install - never composer update as part of environment setup.
11. Copy .env.example to .env if no local file exists.
12. Set APP_ENV=local and a local APP_URL.
13. Set the dedicated local database and Redis configuration.

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

14. Generate APP_KEY only for the local environment if needed.
15. Run migrations against foodonthego_local only.
16. Run the project's controlled development seeders/fixtures only.
17. Clear local caches.
18. Start Laravel on 0.0.0.0 so a physical Android phone can reach it over the LAN.

```
composer install
copy .env.example .env
php artisan key:generate
php artisan optimize:clear
php artisan migrate
# Run only the project's approved dev seed command, for example:
# php artisan db:seed --class=DevelopmentSeeder
php artisan storage:link
php artisan serve --host=0.0.0.0 --port=8000
```

Migration safety

Before running php artisan migrate, display the resolved DB_HOST and DB_DATABASE and prove it is the local database. If the hostname/database resembles production, STOP. Never use migrate:fresh against any unknown or shared database.

8. Backend Local .env Checklist

```
APP_NAME=FoodOnTheGo
APP_ENV=local
APP_DEBUG=true
APP_URL=http://127.0.0.1:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=foodonthego_local
DB_USERNAME=<local_user>
DB_PASSWORD=<local_password>

CACHE_STORE=redis
SESSION_DRIVER=redis
QUEUE_CONNECTION=redis
REDIS_HOST=127.0.0.1
REDIS_PORT=6379

FILESYSTEM_DISK=local
```

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

```
MAIL_MAILER=log

# Use dedicated development/test provider credentials only.
RAZORPAY_KEY_ID=<TEST_KEY_ID>
RAZORPAY_KEY_SECRET=<TEST_SECRET>
GOOGLE_MAPS_SERVER_KEY=<RESTRICTED_DEV_SERVER_KEY>
```

Variable names must match the repository. Do not rename working project variables to match this example.

9. Local Health Endpoint and Readiness Proof

Create or verify a safe local health endpoint if the project does not already have one. It must not reveal secrets.

```
GET http://127.0.0.1:8000/api/health
```

Expected safe response concept:

```
{
  "status": "ok",
  "environment": "local",
  "database": "ok",
  "redis": "ok",
  "version": "<commit/build>"
}
```

- ☐ HTTP 200 from the Windows browser.
- ☐ Database reports healthy.
- ☐ Redis reports healthy.
- ☐ No secrets or connection strings are returned.
- ☐ The same endpoint is reachable from the Android phone before building the review APK.

10. Customer Web - Local Configuration

19. Open the actual Customer Web directory discovered in the repository.
20. Install exactly from the lockfile: `npm ci` (or the project's equivalent package manager).
21. Create a local environment file supported by the framework, such as `.env.local`.
22. Point the Web app only to the local Laravel API.
23. Start the development server.
24. Open the app and verify the browser Network tab shows `localhost/local` API requests - not the `cloud/live` domain.

```
npm ci
# Example only - use actual project variable name:
# VITE_API_BASE_URL=http://127.0.0.1:8000/api/v1
npm run dev
```

Check	Expected
Customer Web URL	http://localhost:5173 or actual project port
API calls	127.0.0.1:8000 / local API only
Console	No functional errors
Network	No requests to production domain unless an explicitly approved public provider is required
Environment indicator	A development-only LOCAL badge is recommended so screenshots cannot be confused with live.

11. CORS / Cookie / Session Configuration

- **Token-auth project:** keep the existing token design; do not switch auth mechanisms during local setup.
- **Cookie/Sanctum project:** allow the exact localhost origins and configure stateful domains/credentials correctly.
- **Never:** use wildcard CORS with credentialed cookies.
- **Physical Android:** native Flutter is not governed by browser CORS, but the API still needs to listen on the correct host/interface.

Conceptual local origins - use actual project config.

http://localhost:5173

http://127.0.0.1:5173

http://localhost:5174

http://localhost:5175

12. Optional Local Workers / Realtime Services

Start only services required by the module currently under test. Do not make the basic Home/Login pages depend on unnecessary background services.

```
php artisan queue:work
```

When realtime modules require it:

```
php artisan reverb:start --host=0.0.0.0 --port=8080
```

- Queue worker: needed when the current flow dispatches jobs that must complete during testing.
- Reverb/WebSockets: start when real-time modules are being tested; not mandatory for basic local bootstrap.
- Scheduler: run only if a tested feature depends on scheduled jobs.

13. Android Toolchain - One-Time Setup

- ☐ Install Flutter SDK and add flutter/bin to PATH.
- ☐ Install Android Studio.

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

- ☐ Install Android SDK Platform required by the project.
- ☐ Install Android SDK Build Tools.
- ☐ Install Android SDK Platform-Tools (adb).
- ☐ Accept Android licenses: flutter doctor --android-licenses.
- ☐ Run flutter doctor -v and resolve Android errors.
- ☐ Connect a physical phone with Developer Options + USB debugging OR use the same private Wi-Fi network.
- ☐ Confirm adb devices sees the phone when using USB.

```
flutter doctor -v
flutter doctor --android-licenses
adb devices
```

14. Android API Connection - Method A: Same Wi-Fi

25. Connect the Windows PC and Android phone to the same trusted private Wi-Fi network.
26. Run ipconfig on Windows and identify the PC IPv4 address for that network, e.g. 192.168.1.50.
27. Start Laravel with --host=0.0.0.0 --port=8000.
28. Allow TCP 8000 on Windows Firewall for Private networks only if required.
29. On the phone browser, open http://<PC-IP>:8000/api/health.
30. Only after health works, set the Android local API base URL to http://<PC-IP>:8000/api/v1.

```
ipconfig
php artisan serve --host=0.0.0.0 --port=8000

# Example Android local API value only:
http://192.168.1.50:8000/api/v1
```

Security

Do not open port 8000 on the Public firewall profile and do not port-forward it on the router. This is a private-LAN development endpoint, not a temporary production server.

15. Android API Connection - Method B: USB adb reverse

Use this when the phone is connected by USB and USB debugging is enabled. It avoids Wi-Fi/firewall/IP changes.

```
adb devices
adb reverse tcp:8000 tcp:8000
adb reverse --list

# Then the Android app can use:
http://127.0.0.1:8000/api/v1
```

The reverse mapping exists while the device/ADB session is active. If the owner installs the APK and tests away from the PC/USB cable, use the same-Wi-Fi method instead.

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

16. Android Emulator Networking

```
# Standard Android emulator -> Windows host
http://10.0.2.2:8000/api/v1
```

- Do not use 127.0.0.1 to reach the PC from the standard emulator.
- If a different emulator vendor is used, confirm its host gateway address.
- Keep emulator API config separate from physical-device local config.

17. Android Debug-Only HTTP / Cleartext Handling

Local development commonly uses HTTP. Android may block cleartext traffic depending on API level/app configuration. Any exception must be limited to the local/debug build; production must remain HTTPS-only.

Required separation

Do not solve local HTTP by globally enabling cleartext traffic in the production manifest. Use a debug/local flavor, debug manifest override, or debug-only network security configuration.

18. Flutter Environment and Build Flavors

Preferred: create explicit environment/flavor separation so local APKs cannot accidentally point to production.

Flavor / mode	Purpose	API
local	Developer PC + manual phone testing	LAN IP / adb reverse / emulator host gateway
staging	Future shared non-production environment	HTTPS staging URL
production	Future live release	HTTPS production URL only

```
# Conceptual use - adapt to the existing Flutter configuration.
flutter run --flavor local \
  --dart-define=APP_ENV=local \
  --dart-define=API_BASE_URL=http://10.0.2.2:8000/api/v1
```

If the project does not currently use flavors, the bootstrap prompt in this guide instructs Claude to implement a minimal safe local configuration without breaking the existing production build.

19. Google Maps / Places - Local Keys

Surface	Key rule
Web	Use a dedicated development browser key restricted to localhost/127.0.0.1 origins and only required APIs.
Android	Use a development Android key restricted to the actual package/applicationId and debug signing fingerprint where supported.
Laravel server	Use a server-side development key restricted appropriately; never ship it in Web or APK.

- Do not reuse an unrestricted production Google key locally.
- Record provider calls during tests; do not trigger Routes API on every page refresh.
- If Maps/Places are not needed for the current module, they may remain disabled until that module.

20. Razorpay, OTP, Firebase and Storage in Local Mode

Integration	Local behavior
Razorpay	TEST mode only. No live charge. Webhooks may use a secure development tunnel later only when payment testing requires it.
OTP/SMS	Use a local/test provider or controlled test OTP. Production master OTP must be impossible.
Firebase FCM	Use a dev Firebase project when push module is tested; otherwise disable cleanly.
S3/uploads	Prefer Laravel local disk during bootstrap. Use dev bucket only when storage integration itself is under test.
Sentry/monitoring	Use a dev project/environment or disable during local bootstrap; never contaminate production monitoring.

21. Local Database Seeding and Test Identities

Controlled fixtures only

Seed only development/test data. Never copy personal production data to local just to make the UI look complete.

Fixture	Purpose
Rahul Sharma	Controlled Customer account
Ananya Sharma	Second Customer for isolation/IDOR/account-switch tests
Highway Spice Kitchen	Controlled Restaurant
Priya Verma	Controlled Restaurant Owner/manager identity when restaurant modules begin
FoodOnTheGo Platform Admin	Controlled Admin identity when admin modules begin
Delhi -> Jaipur	Controlled journey for routing/restaurant discovery tests

Use these only if the project already follows these fixtures or deliberately creates them in a DevelopmentSeeder. Never imply they are production records.

22. Start / Stop / Verify Scripts to Create

The first new-session task should create simple scripts so local startup is repeatable instead of relying on memory.

Script	Purpose
scripts/local/start-backend.ps1	Start/verify MySQL + Redis + Laravel and any required worker.
scripts/local/start-web.ps1	Start Customer Web on the documented port.
scripts/local/verify-local.ps1	Check ports, API health, DB/Redis health and detect any production URL references.
scripts/local/android-info.ps1	Print emulator/phone connection method and resolved API base URL.
scripts/local/build-android-local.ps1	Build the installable local-review APK and generate SHA-256.
scripts/local/stop-local.ps1	Stop only local project processes/containers.

23. Local Web Verification Gate

- ☐ Customer Web opens from localhost.
- ☐ Browser Network panel proves API requests go to local Laravel, not production.
- ☐ Login/session flow works with controlled local/test identity.
- ☐ Home renders real local data or honest empty states.
- ☐ No uncaught JavaScript errors.

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

- ☐ No API 5xx during the tested flow.
- ☐ Responsive checks at 360, 390, 430, 768, 1024, 1280, 1440 and 1920.
- ☐ Screenshots are captured from the local build and labeled LOCAL.
- ☐ The live/cloud website is not used as evidence for local module completion.

24. Android APK Build for Manual Phone Review

After the local Android app works with the local API, produce a dedicated local-review APK. The APK should visibly identify itself as LOCAL/DEV so it cannot be confused with production.

```
flutter clean
flutter pub get
flutter analyze
flutter test

# Preferred when flavors are configured:
flutter build apk --flavor local --debug \
--dart-define=APP_ENV=local \
--dart-define=API_BASE_URL=http://<PC-LAN-IP>:8000/api/v1
```

```
# If the project does not use flavors yet, use the existing safe local
# configuration mechanism and document the exact command.
```

- Use a debug/local-review APK for the first owner test. Do not sign a production release unnecessarily.
- The APK must point to the PC local API, not the cloud API.
- If the PC IP changes, rebuild the LAN-based APK or use a configuration mechanism designed by the project.
- If testing via adb reverse, the APK may use 127.0.0.1:8000 but requires the USB reverse mapping while testing.

25. APK Artifact Naming, Hash and Install

```
# Example artifact naming convention:
FoodOnTheGo-local-review-v0.1.0-build001.apk

# PowerShell SHA-256:
Get-FileHash .\FoodOnTheGo-local-review-v0.1.0-build001.apk -Algorithm SHA256

# Optional USB install:
adb install -r .\FoodOnTheGo-local-review-v0.1.0-build001.apk
```

Record	Required
APK filename	Yes

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

Record	Required
App version/build	Yes
Git commit SHA	Yes
Build environment	LOCAL
API base URL	Safe local value; do not include secrets
SHA-256	Yes
Build command	Yes
Test device + Android version	Yes
Build timestamp	Yes

26. Manual Android Phone Test - Owner Checklist

- ☐ Install APK successfully.
- ☐ Confirm the app visibly indicates LOCAL/DEV.
- ☐ Confirm API health is reachable from the phone.
- ☐ Launch logged out and verify login screen.
- ☐ Authenticate with the controlled local/test account.
- ☐ Verify Home/navigation.
- ☐ Verify no production customer data appears.
- ☐ Background and reopen the app.
- ☐ Kill and reopen the app.
- ☐ Turn Wi-Fi off and confirm a controlled offline/error state.
- ☐ Turn Wi-Fi back on and confirm recovery.
- ☐ Check fonts, spacing, touch targets, scrolling and back button.
- ☐ Record every mismatch with screenshot and screen name.
- ☐ Do not approve live deployment until critical issues are fixed and a new APK is reviewed.

27. Local Evidence Folder

```

/docs/local-review/
  environment/
  screenshots/web/
  screenshots/android/
  test-results/
  logs/
  apk/
  LOCAL-SETUP.md

```

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

LOCAL-READINESS.md

APK-BUILD-MANIFEST.md

BUG-REGISTER.md

FEATURE-TRACEABILITY.md

- Do not put secrets in evidence files.
- Screenshots should show local environment indicators where practical.
- Keep exact commands and test results so the setup is reproducible in a new session.

28. Local Readiness Tree

LOCAL READINESS

|

+-- [] Repository audit

+-- [] Local environment isolation

+-- [] MySQL local database

+-- [] Redis local

+-- [] Laravel API

| +-- [] /api/health

| +-- [] migrations

| +-- [] dev fixtures

| +-- [] logs clean

|

+-- [] Customer Web

| +-- [] localhost only

| +-- [] local API only

| +-- [] responsive

| +-- [] browser console clean

|

+-- [] Android

| +-- [] emulator connection

| +-- [] physical phone connection

| +-- [] local API

| +-- [] local/debug HTTP safely scoped

| +-- [] manual phone test

|

+-- [] APK

| +-- [] build

| +-- [] version

| +-- [] SHA-256

| +-- [] install

| +-- [] owner approval

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

|

+-- [X] iOS skipped for this phase

|

+-- [BLOCKED] LIVE DEPLOYMENT

+-- unlock only after explicit owner approval

29. Hard Stop Conditions

Condition	Required action
Any config points to production DB	STOP immediately; correct environment before migrations/tests.
Customer Web calls live API	STOP; fix API base URL/environment injection.
Android APK calls live API	STOP; rebuild with local config.
Live Razorpay key/charge	STOP; replace with TEST mode.
Server secret found in Web/APK	STOP; rotate if exposed and correct architecture.
Local test modifies cloud/live data	STOP; investigate before continuing.
Critical local flow fails	Do not start the next feature module.
Owner has not approved local Web/APK	Do not proceed to live deployment.

30. First New-Session Prompt for Claude

Use this before feature modules

Paste the following prompt into the new development session. Its only job is to inspect the repository, configure the local environment, prove Web + Android are using local services, and produce the first local-review APK. It must not deploy anything live.

FOODONTHEGO - LOCAL BOOTSTRAP MODULE 00

LOCAL ENVIRONMENT, CUSTOMER WEB, ANDROID CONNECTION & REVIEW APK

NO LIVE/CLOUD DEPLOYMENT

IOS OUT OF SCOPE

We are restarting FoodOnTheGo development in a new session because previous work was being tested directly against the live/cloud environment and the live view did not reliably reflect the expected implementation.

From now on use a strict LOCAL-FIRST workflow.

PRIORITY ORDER:

1. Customer Web application

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

2. Laravel backend/API + MySQL + Redis
3. Android Flutter application
4. Installable Android local-review APK
5. Manual owner testing
6. Only after explicit approval: a separate future live-deployment phase

IOS IS OUT OF SCOPE. Do not configure, build, test, debug or report iOS as a blocker in this phase.

HARD RULE: DO NOT TOUCH THE LIVE ENVIRONMENT.

Do not SSH to production. Do not upload files to production. Do not run production migrations. Do not modify the production database. Do not clear production caches. Do not deploy. Do not use live Razorpay charges. Do not change live DNS. Do not use the cloud/live application as proof that local work is complete.

PHASE A - REPOSITORY AUDIT

- Identify the actual backend, customer web, restaurant web, admin web and Flutter directories.
- Identify composer.json/composer.lock, package manager lockfiles, pubspec.lock, Android Gradle/JDK requirements and existing Docker files.
- Record exact versions/config requirements from the repository. Do not upgrade packages during bootstrap unless required to make the existing project run.
- Locate every .env.example and every API-base-url/environment implementation.
- Search the repository for production domains, production database hosts, live payment keys, hard-coded API URLs and server secrets exposed to clients.
- Produce a LOCAL CONFIG GAP REPORT before changing code.

PHASE B - LOCAL SAFETY

- Create/verify a dedicated local environment only.
- APP_ENV must be local.
- Use a dedicated foodonthego_local database.
- MySQL and Redis must be local.
- Use local filesystem initially, not production S3.
- Use TEST Razorpay only.
- Use development/restricted Google keys only.
- Use controlled development OTP only and make it impossible in production.
- Do not copy production customer data locally.
- Create/verify DevelopmentSeeder with safe controlled fixtures only where needed.

Before ANY migration, print and verify the resolved DB host/database. If it resembles production, STOP.

PHASE C - START LOCAL BACKEND

- Install dependencies from lockfiles: composer install, never composer update for bootstrap.
- Configure backend .env from the project's variables.
- Run local migrations and approved development seeders only.
- Start Laravel locally on 0.0.0.0:8000 so a physical Android phone can reach it.
- Start Redis and queue worker when the current flow needs it.
- Start Reverb only if current tested functionality needs realtime.

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

- Create/verify GET /api/health with safe status only: app, environment, DB, Redis, build/commit. Never return secrets.
- Verify browser can reach http://127.0.0.1:8000/api/health.

PHASE D - CUSTOMER WEB LOCAL

- Configure Customer Web to call ONLY the local Laravel API.
- Run the actual Customer Web locally.
- Record the actual localhost URL/port.
- Open it in a browser and inspect Network + Console.
- PROVE no application request is going to the production FoodOnTheGo API/domain.
- Add a development-only LOCAL environment badge/watermark if one does not exist, so screenshots can never be mistaken for live.
- Do not implement feature modules yet. The bootstrap only needs the existing app shell to run correctly.

PHASE E - ANDROID LOCAL NETWORKING

Support and document all three local endpoint patterns:

1. Android emulator -> http://10.0.2.2:8000/api/v1
2. Physical phone same Wi-Fi -> http://<PC-LAN-IP>:8000/api/v1
3. USB development -> adb reverse tcp:8000 tcp:8000, then http://127.0.0.1:8000/api/v1

For physical Wi-Fi testing:

- identify the PC IPv4 address
- Laravel must listen on 0.0.0.0:8000
- if needed, allow port 8000 only on the Windows Private firewall profile
- verify the phone browser can open http://<PC-LAN-IP>:8000/api/health before running the Flutter app

Do not open the development API to the public internet.

PHASE F - ANDROID ENVIRONMENT SEPARATION

- Audit current Flutter environment configuration.
- Prefer/create a safe local flavor/config without breaking production.
- Local/debug build may allow local HTTP only through a debug/local-scoped Android configuration.
- Do NOT globally enable cleartext traffic for production.
- Ensure local API base URL is injected by the project's configuration mechanism and is visible in a safe Build Information screen/log (without secrets).
- Add a visible LOCAL/DEV indicator to the local Android build.
- No iOS work.

PHASE G - GOOGLE / RAZORPAY / OTP SECURITY

- Web Google key: development key restricted to localhost/127.0.0.1 and required APIs.
- Android Google key: development key restricted to package/applicationId + debug signature where applicable.
- Server Google key: server-only; never in Web bundle/APK.
- Razorpay: TEST mode only; secret server-only.
- Development OTP/test provider: local/staging only; production bypass must be impossible.
- Run a secret scan.

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

PHASE H - CREATE REPEATABLE POWERSHELL SCRIPTS

Create or update:

- scripts/local/start-backend.ps1
- scripts/local/start-web.ps1
- scripts/local/verify-local.ps1
- scripts/local/android-info.ps1
- scripts/local/build-android-local.ps1
- scripts/local/stop-local.ps1

verify-local.ps1 must at least check:

- expected local ports
- Laravel /api/health
- DB/Redis health
- Customer Web URL
- production-domain references in resolved runtime config
- Android API mode/config

and must FAIL if it detects a production database or live API base URL in local mode.

PHASE I - TEST BEFORE APK

Run actual tests that exist in the repository:

- Laravel tests
- Customer Web tests
- Flutter analyze
- Flutter unit/widget tests
- relevant integration/smoke tests

Do not write PASS unless the command actually ran.

Report exact command, passed, failed, skipped and blocker.

Open the local Customer Web and Android app and visually inspect the actual existing shell/login/home. Capture local screenshots.

PHASE J - BUILD ANDROID LOCAL-REVIEW APK

Only after local API + Customer Web + Android are connected:

- build a LOCAL/DEBUG review APK using the repository's correct flavor/config
- it must point to the local API, never production
- record APK filename
- app version/build
- Git commit SHA
- build command
- API mode/base URL (safe value only)
- build timestamp
- SHA-256
- device/emulator used for verification

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

Prefer a name such as:

FoodOnTheGo-local-review-<version>-<build>.apk

Install the exact generated APK on an Android device/emulator and verify it launches and reaches the local backend.

PHASE K - DOCUMENTATION

Create/update:

/docs/local-review/LOCAL-SETUP.md

/docs/local-review/LOCAL-READINESS.md

/docs/local-review/APK-BUILD-MANIFEST.md

/docs/local-review/BUG-REGISTER.md

/docs/local-review/FEATURE-TRACEABILITY.md

/docs/local-review/LIVE-DEPLOYMENT-GATE.md

Create evidence folders for local Web and Android screenshots/test results.

LOCAL-READINESS.md must show this tree with real statuses:

LOCAL READINESS

- | - Repository
- | - Local DB/Redis
- | - Laravel API
- | - Customer Web
- | - Android emulator/device
- | - APK
- | - Manual owner review
- | - iOS SKIPPED
- `- Live deployment BLOCKED

PHASE L - COMPLETION GATE

LOCAL BOOTSTRAP MODULE 00 can be COMPLETE only if:

- no local runtime points to the production database
- Customer Web runs locally and uses local API
- Laravel runs locally
- MySQL and Redis are healthy
- Android runs against the local API
- physical phone access method is verified or precisely documented
- local-only HTTP behavior is safely debug-scoped
- TEST/dev provider keys are used
- tests were actually run
- local screenshots exist
- APK was actually generated
- SHA-256 was generated
- exact APK was installed/tested on Android if a device is available
- iOS was skipped

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

- live deployment was NOT performed

If any critical item is missing, status is NOT COMPLETE.

FINAL REPORT FORMAT

- A. Final Status
- B. Repository Paths
- C. Tool/Version Audit
- D. Local Environment Variables (names/status only, no secrets)
- E. Local DB/Redis
- F. Laravel Local API + Health
- G. Customer Web URL + API proof
- H. Production Reference Scan
- I. Android Emulator Configuration
- J. Physical Android Phone Configuration
- K. Firewall / Connectivity
- L. Android Local Flavor / HTTP Safety
- M. Google Key Separation
- N. Razorpay Test Configuration
- O. OTP Local Configuration
- P. Scripts Created
- Q. Tests Actually Run
- R. Local Screens Visually Verified
- S. APK Build
- T. APK SHA-256
- U. APK Install/Launch Result
- V. Bugs Found
- W. Bugs Fixed
- X. Remaining Blockers
- Y. Documentation Created
- Z. LOCAL READINESS TREE
- AA. Live Deployment Gate = BLOCKED
- AB. Files Changed
- AC. Exact next input needed for Restart Module 01 feature work

Finally state exactly one:

LOCAL BOOTSTRAP MODULE 00 = COMPLETE

or

LOCAL BOOTSTRAP MODULE 00 = NOT COMPLETE

STOP.

DO NOT START FEATURE MODULES.

DO NOT DEPLOY LIVE.

LOCAL DEVELOPMENT ONLY - Do not deploy to production until manual approval

31. What Comes After Local Bootstrap

31. You manually open the Customer Web on localhost and verify the current shell/login/home.
32. You install the generated local-review APK on your Android phone and verify it can reach the PC local API.
33. Any bootstrap/network/environment issue is fixed before feature modules start.
34. Then restart feature prompts one by one: Home, Authentication, Profile, Trip Planner, Route, Restaurant Listing, Menu, Cart, Pickup, Checkout, Payment, Confirmation, Tracking, ETA, etc.
35. Each feature prompt tests Web + Android locally and updates the same local evidence set.
36. Only after all agreed modules pass and you approve the Web + APK do we create a separate live deployment plan.

Final rule

LOCAL FIRST. WEB + ANDROID ONLY. MANUAL APK APPROVAL. LIVE DEPLOYMENT LATER AND ONLY BY EXPLICIT APPROVAL.